



PROJECT-BASED LEARNING

PHASE-I EVALUATION

**Secure CBT: Hierarchical Key-Chained
Question Paper Delivery & Leak-Traceability System**

Team ID: DSCPP-III-2026-T145

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026-27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T145
Team Name: Secure CBT
Semester: 3rd
Section: BTECH

Team Members

1. Naman – 2510013438
2. Aayush Gupta – 2510370185
3. Kushagra Bhushan – 2510012263

Mentor

Mr. Kartikey Arora

Interactions completed: 2

Problem Statement & Motivation

Problem Statement

Question-paper leaks recur across India's education and recruitment exams at both state and national level, often forcing delayed or repeated exams. This project treats leak prevention as a systems-security problem: build a CBT pipeline where leaking is hard by design, and where any leak that does occur can be traced back to its source.

Motivation

- ▶ Paper leaks delay results and force costly retests
- ▶ Erodes public trust in exam systems
- ▶ Existing security lacks built-in traceability

Target Users / Stakeholders

- ▶ Exam boards and testing agencies (e.g. NTA)
- ▶ Exam centres, teachers, and candidates
- ▶ Regulators investigating leak incidents

Objectives

Project Objectives

1. Prevent leaks at the three vulnerable stages: paper setting, distribution, and the exam centre.
2. Ensure every candidate receives an identical set of questions once unlocked (fairness).
3. Guarantee that any leaked ciphertext can be traced to its exact source node, without a separate fingerprint database.
4. Implement hierarchical XOR key-chaining, AES-256 encryption, and Huffman compression for secure delivery.
5. Demonstrate tamper detection via a SHA-256 hash-chained transfer log and time-locked decryption.

Key Objective: Build traceability directly into the encryption chain – a leaked file is matched to its issuing node via the AVL registry, with no post-hoc log forensics needed.

Proposed Solution

Proposed Approach

1. Teachers submit questions independently; PaperGenerator assembles a balanced paper
2. Huffman compression of the assembled byte stream
3. Hierarchical XOR key-chaining + AES-256 encryption per node
4. AVL tree registry of node secrets, keys, status
5. Time-locked decryption at the centre

Each node's ciphertext is uniquely keyed by its position in the hierarchy, so a leaked file is matched directly against the AVL registry to reveal its source.

Key Features

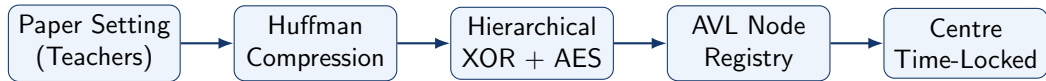
- ▶ No single teacher controls final assembly
- ▶ Each node gets a uniquely encrypted copy
- ▶ BFS broadcast down an LCRS N-ary tree
- ▶ SHA-256 hash-chained transfer log
- ▶ Dijkstra routing + MST backbone design

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	Trees (LCRS N-ary tree), AVL trees, hashing (SHA-256 chain), min-heaps, graphs	Hierarchy & registry (Phase 2)
OOPs with C++	Classes for QuestionBank, Teacher, PaperGenerator; encapsulation of node secrets	Paper setting & node modeling
Algorithms	Huffman coding, BFS broadcast, Dijkstra's algorithm, Minimum Spanning Tree	Compression, distribution, routing
Security / Cryptography	AES-256 encryption, XOR key-chaining, SHA-256 hashing	Encryption & integrity logging

Requirement: Demonstrate meaningful integration of both subjects in **one** project.

System Architecture / Workflow



Major Components

- ▶ QuestionBank, Teacher, PaperGenerator (Phase 1)
- ▶ LCRS N-ary tree + AVL registry (Phase 2)
- ▶ Time-lock module + AVL leak-tracing (Phase 3)

Data / Control Flow

- ▶ Paper flows strictly parent → child via BFS
- ▶ Each node decrypts using parent key + own secret
- ▶ Centre releases paper only after scheduled slot

Technology Stack & Methodology

Technology Stack

- ▶ Programming: C++17
- ▶ Cryptography: OpenSSL (AES-256, SHA-256)
- ▶ Data Structures: LCRS tree, AVL tree, min-heap, graph
- ▶ Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis
2. Class & Data-Structure Design
3. Implementation (Phase 1 → 3)
4. Testing (tamper detection, leak tracing)
5. Evaluation & Reporting

C++ was chosen for fine-grained control over data structures and crypto integration; OpenSSL provides standards-compliant AES/SHA-256 primitives (NIST FIPS 197, FIPS 180-4).

Development Methodology – Detailed

1. Requirement Analysis

Studied existing CBT systems to find gaps, then defined our core requirements: leak prevention, fairness, and built-in traceability.

2. Class & Data-Structure Design

Mapped each phase to a structure – LCRS tree, AVL registry, hash-chain, min-heap – before finalizing the class layout.

3. Implementation (Phase 1 → 3)

Built modules in run order: compression first, then hierarchical encryption and distribution, then time-locked release.

4. Testing

Broke the hash-chain to confirm tamper detection, and traced a simulated leaked file back to its node via the AVL registry.

5. Evaluation & Reporting

Measured lookup performance and tracing accuracy across a multi-centre simulation, then compiled the final report.

Each step feeds directly into the next – design decisions map one-to-one onto later implementation and test modules.

Initial Progress & Team Contribution

Progress Achieved

- ▶ Hierarchical key-chaining scheme finalized
- ▶ Class/data-structure design complete for all 3 phases
- ▶ QuestionBank, Teacher, PaperGenerator implemented
- ▶ Huffman compression module in progress
- ▶ LCRS tree, AVL registry, and encryption – pending

Individual Contribution

Member	Role	Contribution
Naman	Team Lead	<i>core implementation</i>
Kushagra Bhushan	Developer	<i>Architecture & security design</i>
Aayush Gupta	Developer	<i>Support and documentation</i>

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal & Design
2. Phase-II: Development (compression, key-chaining, AVL, hash-chain, time-lock)
3. Phase-III: Full integration, Dijkstra/MST layer, evaluation & demo

Expected Outcomes

- ▶ End-to-end working CBT security simulation
- ▶ Demonstrated tamper detection via hash-chain
- ▶ Demonstrated leak-source tracing via AVL registry

References

1. Newslaundry (2024), Indian paper-leak data journalism
2. The Wire, "India's Exam Fraud Bubble" (148 cases, 1 conviction)
3. Cormen et al., *Introduction to Algorithms*
4. NIST FIPS 197 (AES) & FIPS 180-4 (SHA-256)

Thank You
Questions & Suggestions